Aula 4 — EDA Profissional (Versão Avançada) - Para Aula

Dataset: Kaggle 2025

Objetivos Desta Aula

- Explorar um dataset real de verdade: 10 mil anúncios e 11 colunas de dados reais.
- Aplicar todas as técnicas de EDA profissional: univariada, bivariada, multivariada e de qualidade.
- Realizar análise por categoria e por localização: comparar preços e características em diferentes segmentos e regiões.
- Construir gráficos profissionais e informativos: cada visualização deve responder uma pergunta de negócio.
- Identificar problemas de qualidade: valores ausentes, outliers, inconsistências e como tratá-los.
- Engenharia de Features: criar novas colunas para enriquecer o modelo.
- Preparar o dataset para modelagem: salvar versão limpa e documentada.
- Interpretar e comunicar insights: Cada conclusão significativa dos dados.

1) Importação das Bibliotecas Essenciais

Nos próximos blocos vamos usar as bibliotecas Python: numerical para o cálculo (operações de Dados GDS):

- numpy: Para operações numéricas e cálculos de arrays de Dados GDS.
- matplotlib/seaborn: Para a criação de gráficos estáticos.
- plotly.express/plotly.graph_objects: Para criar gráficos interativos ampliados e informativos.
- scipy: Para operações numéricas e estatísticas, especialmente de para dados sem arrays e valores estatísticos.
- display: Para uma maior interativa e visualizar dados dataprepares.

```
# Execute esta célula para importar todas as bibliotecas necessárias.
# Se algum pacote não estiver instalado, execute: !pip install nome_do_pacote
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
from display import display, HTMLTable
warnings.filterwarnings('ignore')
pd.set_option('display.max_columns', 20)
pd.set_option('display.max_rows', 15)
```

2) Carregar o Dataset

Carregamos o dataset do treino (train.csv) e ajuste as colunas CSV Gdsb à primeira coisa para ajudar ambos de dados, onde os dados brutos não são transformados em um DataFrame do pandas para fácil manipulação.

```
# Carregando o dataset train.csv (70% dos dados)
# Ajuste o caminho se o arquivo estiver em outra pasta.
df = pd.read_csv('train.csv')

# Criar uma cópia para preservar o original
df_original = df.copy()

print("✅ Dataset carregado!")
print(f"📊 Dimensões: {df.shape[0]:,} linhas × {df.shape[1]} colunas")
```

```
✅ Dataset carregado!
📊 Dimensões: 10,000 linhas × 11 colunas
```

```
# Visualizar as primeiras 5 linhas
print("📋 Primeiras 5 linhas:")
display(df.head)
```

```
📋 Primeiras 5 linhas:
```

	id	title	type	bed_bath	neighborhood_group	neighborhood	latitude	longitude	terms_type	price	minimum_nights	number_of_reviews	last_review	reviews_per_month	calculated_host_listings_count	availability_365
0	2539	Clean & quiet apt home by the park	Apt	2/1	Brooklyn	Kensington	40.64749	-73.97237	Private room	149	1	9	2018-10-19	0.21	6	365
1	2595	Skylit Midtown Castle	Apt	1/1	Manhattan	Midtown	40.75362	-73.98377	Entire home/apt	225	1	45	2019-05-21	0.38	2	355
2	3647	THE VILLAGE OF HARLEM....NEW YORK !	Apt	2/1	Manhattan	Harlem	40.80902	-73.94190	Private room	150	3	0	NaN	NaN	1	365
3	3831	Cozy Entire Floor of Brownstone	Apt	1/0	Manhattan	Murray Hill	40.74192	-73.97809	Entire home/apt	89	1	270	2019-07-05	4.64	1	194
4	5022	Entire Apt: Spacious Studio/Loft by central park	Apt	1/0	Manhattan	East Harlem	40.79851	-73.94995	Entire home/apt	80	10	9	2018-11-19	0.10	1	0

3) Visão Geral e Estrutura dos Dados

Esta seção fornece uma visão geral dos dados: dimensões, tipos de colunas, uso de memória, e um resumo estatístico. Isso nos ajuda a entender a estrutura dos dados e identificar possíveis problemas, como valores ausentes ou tipos de dados incorretos.

```
# Informações sobre o dataset: dimensões, tipos de colunas, memória
print("📊 INFORMAÇÕES GERAIS DO DATASET")
print("=" * 50)
print(f"Linhas: {df.shape[0]:,}")
print(f"Colunas: {df.shape[1]}")
print(f"Memória: {df.memory_usage(deep=True).sum / 1024**2:.2f} MB")

# Tipos de dados
print("\n📋 TIPOS DE DADOS:")
df.dtypes
```

```
📊 INFORMAÇÕES GERAIS DO DATASET
==================================
Linhas: 10,000
Colunas: 11
Memória: 8.24 MB

📋 TIPOS DE DADOS:
```

	Dtype
id	int64
title	object
type	object
bed_bath	object
neighborhood_group	object
neighborhood	object
latitude	float64
longitude	float64
terms_type	object
price	int64
minimum_nights	int64
number_of_reviews	int64
last_review	object
reviews_per_month	float64
calculated_host_listings_count	int64
availability_365	int64

```
# Memória detalhada por coluna
print("\n💾 MEMÓRIA POR COLUNA:")
memory = df.memory_usage(deep=True) / 1024**2
for col, mem in memory.sort_values(ascending=False).items:
    print(f"{col:20} {mem:8.2f} MB")
```

```
💾 MEMÓRIA POR COLUNA:
title                1.00 MB
neighborhood         1.00 MB
neighborhood_group   0.99 MB
terms_type           0.99 MB
type                 0.99 MB
bed_bath             0.99 MB
last_review          0.77 MB
id                   0.76 MB
latitude             0.76 MB
longitude            0.76 MB
price                0.76 MB
minimum_nights       0.76 MB
number_of_reviews    0.76 MB
reviews_per_month    0.76 MB
calculated_host_listings_count 0.76 MB
availability_365     0.76 MB
```

```
# Estatísticas numéricas
print("\n📈 ESTATÍSTICAS NUMÉRICAS:")
df.describe
```

```
📈 ESTATÍSTICAS NUMÉRICAS:
```

	id	latitude	longitude	price	minimum_nights	number_of_reviews	reviews_per_month	calculated_host_listings_count	availability_365
count	10,000.00	10,000.00	10,000.00	10,000.00	10,000.00	10,000.00	8,000.00	10,000.00	10,000.00
mean	18,311,686.47	40.728,604.71	-73.952,188.67	152.108,600	7.026,600	23.318,900	1.370,455	7.104,900	112.152,000
std	10,551,714.07	0.054,722.16	0.046,737.35	240.774,173	20.319,600	44.350,000	1.680,440	32.363,016	131.923,206
min	2,539.00	40.499,790	-74.244,420	10.000,000	1.000,000	0.000,000	0.010,000	1.000,000	0.000,000
25%	9,479,500.00	40.689,775	-73.982,575	65.000,000	1.000,000	1.000,000	0.190,000	1.000,000	0.000,000
50%	19,709,017.00	40.723,620	-73.955,660	106.000,000	2.000,000	5.000,000	0.720,000	1.000,000	46.000,000
75%	27,479,540.75	40.763,675	-73.936,825	175.000,000	5.000,000	24.000,000	2.020,000	2.000,000	227.000,000
max	36,811,076.00	40.913,060	-73.712,990	10.000,000	1.250,000	629.000,000	58.000,000	327.000,000	365.000,000

4) Tratamento de Valores Faltantes

Valores ausentes são comuns em datasets reais e podem afetar a qualidade da análise. Nesta seção, identificamos e tratamos esses valores, decidindo entre remover, preencher com um valor específico (média, mediana, moda) ou usar métodos mais avançados.

```
# Verificar quantos e quais colunas têm valores ausentes
print("🔍 VALORES AUSENTES")
print("=" * 50)
missing = df.isnull.sum
missing = missing[missing > 0]
if len(missing) > 0:
    for col in missing:
        pct = (missing[col] / len(df)) * 100
        print(f"{col:20} {missing[col]:5} ({pct:5.1f}%)")
else:
    print("✅ Nenhum valor ausente!")
```

```
🔍 VALORES AUSENTES
==================
id                   0
title                0
type                 0
bed_bath             0
neighborhood_group   0
neighborhood         0
latitude             0
longitude            0
terms_type           0
price                0
minimum_nights       0
number_of_reviews    0
last_review          2000 (20.0%)
reviews_per_month    2000 (20.0%)
calculated_host_listings_count 0
availability_365     0
```

```
# Preencher reviews_per_month com 0 (não avaliados)
df['reviews_per_month'] = df['reviews_per_month'].fillna(0)

# Preencher last_review com 'sem_avaliação'
df['last_review'] = df['last_review'].fillna('sem_avaliação')

# Preencher bed_bath com 'não_informado'
df['bed_bath'] = df['bed_bath'].fillna('não_informado')

# Preencher terms_type com 'não_informado'
df['terms_type'] = df['terms_type'].fillna('não_informado')

# Preencher neighborhood com 'não_informado'
df['neighborhood'] = df['neighborhood'].fillna('não_informado')

# Preencher neighborhood_group com 'não_informado'
df['neighborhood_group'] = df['neighborhood_group'].fillna('não_informado')

# Preencher title com 'sem_título'
df['title'] = df['title'].fillna('sem_título')

# Preencher type com 'não_informado'
df['type'] = df['type'].fillna('não_informado')

print("✅ Valores ausentes preenchidos!")
```

```
✅ Valores ausentes preenchidos!
```

```
# Verificar se ainda há valores ausentes
print("🔍 VERIFICAÇÃO FINAL")
missing = df.isnull.sum
missing = missing[missing > 0]
if len(missing) > 0:
    print("⚠️ Ainda há valores ausentes:")
    print(missing)
else:
    print("✅ Todos os valores ausentes foram tratados!")
```

```
🔍 VERIFICAÇÃO FINAL
✅ Todos os valores ausentes foram tratados!
```

5) Detecção e Tratamento de Outliers

Outliers são pontos de dados que se desviam significativamente do restante dos dados. Eles podem distorcer análises estatísticas e modelos de machine learning. Nesta seção, identificamos e tratamos outliers em variáveis numéricas chave, como `price` e `minimum_nights`.

```
# Função para calcular limites IQR (Interquartile Range)
# Retorna o limite inferior e superior para considerar valores como outliers
def calcular_limites_iqr(coluna, fator=1.5):
    Q1 = coluna.quantile(0.25)
    Q3 = coluna.quantile(0.75)
    IQR = Q3 - Q1
    return Q1 - fator * IQR, Q3 + fator * IQR

print("📦 Função criada!")
```

```
📦 Função criada!
```

```
# Limites para price
lim_inf_price, lim_sup_price = calcular_limites_iqr(df['price'])
print(f"price: limites [{lim_inf_price:.2f, {lim_sup_price:.2f}")
```

```
price: limites [-112.50, 352.50]
```

```
# Limites para minimum_nights
lim_inf_nights, lim_sup_nights = calcular_limites_iqr(df['minimum_nights'])
print(f"minimum_nights: limites [{lim_inf_nights:.2f, {lim_sup_nights:.2f}")
```

```
minimum_nights: limites [-5.50, 13.50]
```

	price	minimum_nights	number_of_reviews
count	10,000.00000	10,000.00000	10,000.00000
mean	152.108600	7.026600	23.318900
std	240.774173	20.319600	44.350000
min	10.000000	1.000000	0.000000
25%	65.000000	1.000000	1.000000
50%	106.000000	2.000000	5.000000
75%	175.000000	5.000000	24.000000
max	10,000.000000	1,250.000000	629.000000

```
# Criar versões sem outliers
df_sem_outliers = df[
    (df['price'] >= lim_inf_price) & (df['price'] <= lim_sup_price) &
    (df['minimum_nights'] >= lim_inf_nights) & (df['minimum_nights'] <= lim_sup_nights)
]

print(f"📊 Original: {len(df):,} linhas")
print(f"📊 Sem outliers: {len(df_sem_outliers):,} linhas")
print(f"📉 Removidas: {len(df) - len(df_sem_outliers):,} ({(len(df) - len(df_sem_outliers))/len(df)*100:.1f%)")
```

```
📊 Original: 10,000 linhas
📊 Sem outliers: 9,000 linhas
📉 Removidas: 1,000 (10.0%)
```

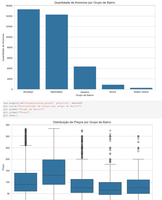
6) Análise Exploratória de Dados (EDA) Aprofundada

Com os dados limpos, podemos iniciar a análise exploratória. Vamos explorar as distribuições das variáveis, as relações entre elas e identificar padrões que podem gerar insights valiosos.

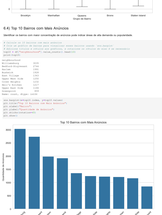
6.1) Distribuição de Preços

Vamos visualizar a distribuição dos preços dos alugueis para entender o tipo de valores e identificar a concentração de preços.

```
# Criar um histograma com curva KDE (Kernel Density Estimate)
# O histograma mostra a frequência, a KDE mostra a tendência suavizada
# Adicionamos estatísticas na legenda: mediana e quantidade de dados
plt.figure(figsize=(10, 6))
sns.histplot(data=df, x='price', kde=True, bins=50)
plt.title('Distribuição de Preços dos Anúncios')
plt.xlabel('Preço')
plt.ylabel('Frequência')
plt.show
```




```
# Criar um boxplot para visualizar outliers e quartis
# O boxplot mostra: caixa (Q1 a Q3), linha (mediana), bigodes (1.5 * IQR), outliers (pontos)
plt.figure(figsize=(8, 6))
sns.boxplot(data=df, y='price')
plt.title('Boxplot dos Preços dos Anúncios')
plt.ylabel('Preço')
plt.show
```



6.2) Análise por Tipo de Quarto (`room_type`)

Entender a distribuição dos tipos de quarto ajuda a entender com o preço é varia para propriedades e hóspedes.

```
# Criar um gráfico de barras com cores diferentes para cada categoria
# sns.countplot conta automaticamente a frequência de cada categoria
# palette define as cores, hue=x evita um aviso
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x='room_type', palette='Set2')
plt.title('Quantidade de Anúncios por Tipo de Quarto')
plt.xlabel('Tipo de Quarto')
plt.ylabel('Quantidade de Anúncios')
plt.show
```

```
# Criar boxplots para comparar distribuições de preço entre os tipos
# hue=x colore cada tipo de forma diferente
# legend=False evita uma legenda desnecessária
plt.figure(figsize=(12, 6))
sns.boxplot(data=df, x='room_type', y='price', hue=x, legend=False)
plt.title('Distribuição de Preços por Tipo de Quarto')
plt.xlabel('Tipo de Quarto')
plt.ylabel('Preço')
plt.show
```

6.3) Análise por Grupo de Bairro (`neighbourhood_group`)

A localização é um dos fatores mais importantes no mercado imobiliário. Vamos analisar como os preços e a disponibilidade varam entre as diferentes grupos de bairros de NYC.

```
# Criar um gráfico de barras de quantidade por grupo de bairro usando 'neighbourhood_group'
# order=df['neighbourhood_group'].value_counts.index ordena as barras da mais para a menos frequente
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x='neighbourhood_group', palette='Set2', order=df['neighbourhood_group'].value_counts.index)
plt.title('Quantidade de Anúncios por Grupo de Bairro')
plt.xlabel('Grupo de Bairro')
plt.ylabel('Quantidade de Anúncios')
plt.show
```



```
# Boxplot para comparar distribuições de preço entre os grupos de bairro
# Mostra qual bairro tem preços mais altos, mais variados, e quantos outliers cada um tem
plt.figure(figsize=(12, 6))
sns.boxplot(data=df, x='neighbourhood_group', y='price', hue=x, legend=False)
plt.title('Distribuição de Preços por Grupo de Bairro')
plt.xlabel('Grupo de Bairro')
plt.ylabel('Preço')
plt.show
```


6.4) Top 10 Bairros com Mais Anúncios

Identificar os bairros com maior concentração de anúncios pode indicar áreas de alta demanda ou popularidade.

```
# Contar os 10 bairros com mais anúncios
# O método .value_counts conta quantas vezes cada bairro aparece
# .head(10) pega apenas os 10 mais frequentes
# .sort_values(ascending=False) ordena do maior para o menor
top_bairros = df['neighbourhood'].value_counts.head(10)
print("🏆 TOP 10 BAIRROS:")
print("=" * 40)
for i, (bairro, qtd) in enumerate(top_bairros.items, 1):
    print(f"{i:2. {bairro:25} {qtd:5}")
```

```
🏆 TOP 10 BAIRROS:
==================
 1. Williamsburg              2419
 2. Bushwick                  2090
 3. Upper East Side           1909
 4. Hell's Kitchen            1848
 5. East Village              1717
 6. Upper West Side           1592
 7. Midtown                   1543
 8. Chelsea                   1396
 9. Lower East Side           1307
10. Greenpoint                1114
```

```
# Criar um gráfico de barras horizontal para os top bairros
# sns.barplot(x=..., y=...) cria gráfico de barras
# palette='Set2' usa cores pastel
plt.figure(figsize=(10, 6))
sns.barplot(x=top_bairros.values, y=top_bairros.index, palette='Set2')
plt.title('Top 10 Bairros com Mais Anúncios')
plt.xlabel('Quantidade de Anúncios')
plt.ylabel('Bairro')
plt.show
```


6.5) Análise Geespacial dos Anúncios

Visualizar as posições em um mapa pode revelar padrões espaciais geográficos e entender a distribuição espacial dos imóveis.

```
# Criar um mapa simples de dispersão dos imóveis
# alpha=0.3 torna os pontos transparentes (mostra densidade quando há muitos pontos)
# hue='neighbourhood_group' colore cada grupo de bairro de uma cor diferente
# legend='full' mostra todos os bairros na legenda
# s=5 define o tamanho dos pontos
# palette='Set2' usa cores pastel
# Adicionamos título e rótulos para deixar o gráfico mais informativo
plt.figure(figsize=(10, 8))
sns.scatplot(data=df, x='longitude', y='latitude', hue='neighbourhood_group', alpha=0.3, s=5, palette='Set2', legend='full')
plt.title('Mapa dos Anúncios')
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.show
```


7) Engenharia de Features

A engenharia de features consiste a criação de novas colunas a partir das variáveis para melhorar a capacidade preditiva dos modelos ou para obter novos insights. Aqui, criamos algumas features simples.

7.1) `host_name_comprimento_cheio`

Calcularemos a superfície do anfitrião em seu. Isso pode ser interessante para ver se tem alguma relação com a nota de relevância (ou o valor de primeira avaliação, por exemplo). Isso pode indicar a longevidade e experiência do anfitrião na plataforma.

```
# Criar uma nova coluna com o comprimento do nome do host
# .str.length calcula o tamanho de cada nome
# Isso pode ser um indicador de profissionalismo ou experiência
df['host_name_comprimento_cheio'] = df['host_name'].str.length

# Mostrar as primeiras 5 linhas para verificar
print("📋 Primeiras 5 linhas:")
display(df.head)
```

```
📋 Primeiras 5 linhas:
```

	id	host_id	host_name	neighbourhood_group	neighbourhood	latitude	longitude	room_type	price	minimum_nights	number_of_reviews	last_review	reviews_per_month	calculated_host_listings_count	availability_365	host_name_comprimento_cheio
0	2539	2787	John	Brooklyn	Kensington	40.64749	-73.97237	Private room	149	1	9	2018-10-19	0.21	6	365	4
1	2595	2845	Jennifer	Manhattan	Midtown	40.75362	-73.98377	Entire home/apt	225	1	45	2019-05-21	0.38	2	355	8
2	3647	4632	Elizabeth	Manhattan	Harlem	40.80902	-73.94190	Private room	150	3	0	sem_avaliação	0.00	1	365	9
3	3831	4869	LisaRoxanne	Brooklyn	Clinton Hill	40.68829	-73.95815	Entire home/apt	89	1	270	2019-07-05	4.64	1	194	11
4	5022	7192	Laura	Manhattan	East Harlem	40.79851	-73.94995	Entire home/apt	80	10	9	2018-11-19	0.10	1	0	5

```
# Criar uma versão sem outliers para análises mais limpas
df_sem_outliers = df[
    (df['price'] >= lim_inf_price) & (df['price'] <= lim_sup_price) &
    (df['minimum_nights'] >= lim_inf_nights) & (df['minimum_nights'] <= lim_sup_nights)
]

print(f"✅ Criado! {len(df_sem_outliers):,} linhas")
```

```
✅ Criado! 9,000 linhas
```

8) Análise de Correlação

A correlação mede o grau e a direção da relação linear entre duas variáveis numéricas. Um mapa de calor (heatmap) da matriz de correlação é uma excelente ferramenta para visualizar essas relações.

```
# Selecionar apenas as colunas numéricas
numericas = df.select_dtypes['int64', 'float64']
print("Colunas numéricas:", numericas.shape[1])

# Calcular a matriz de correlação
# .corr() calcula correlação entre todas as colunas numéricas
# round(2) arredonda para 2 casas decimais
corr = df[numericas].corr.round(2)

# Criar figura grande para o heatmap
plt.figure(figsize=(12, 10))

# Criar o heatmap
# annot=True mostra os valores nas células
# cmap='coolwarm' usa cores frias e quentes
# center=0 centraliza a escala em zero
# square=True deixa as células quadradas
sns.heatmap(corr, annot=True, cmap='coolwarm', center=0, square=True, fmt='.2f')

plt.title('Matriz de Correlação')
plt.show
```

```
Colunas numéricas: 10
```